

## Unit-5

### Machine learning:

Machine learning is a sub field of Artificial Intelligence that provides computer the ability to learn and improve its learning from experience without being written rules explicitly. The machine start learning with observations or data, in order to look for patterns in data and make better decisions in the future based on the examples that it has been provided.

#### Applications:

- Computer Vision Processing
- Natural Language Processing
- Forecasting (e.g. stock market trends)
- Games
- Expert Systems.

The difference between traditional programming and machine learning can be illustrated with following figures:

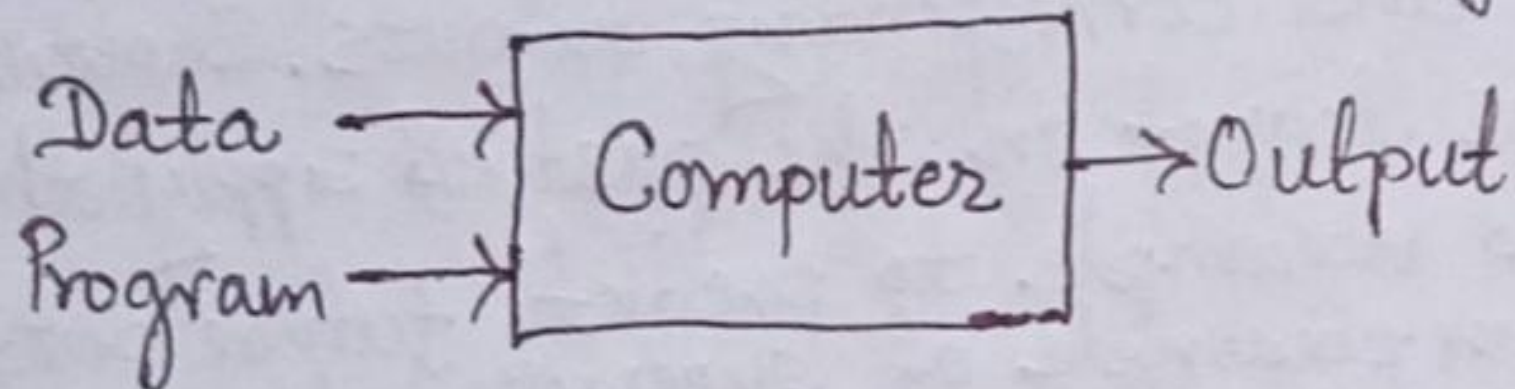


fig: Traditional Programming.

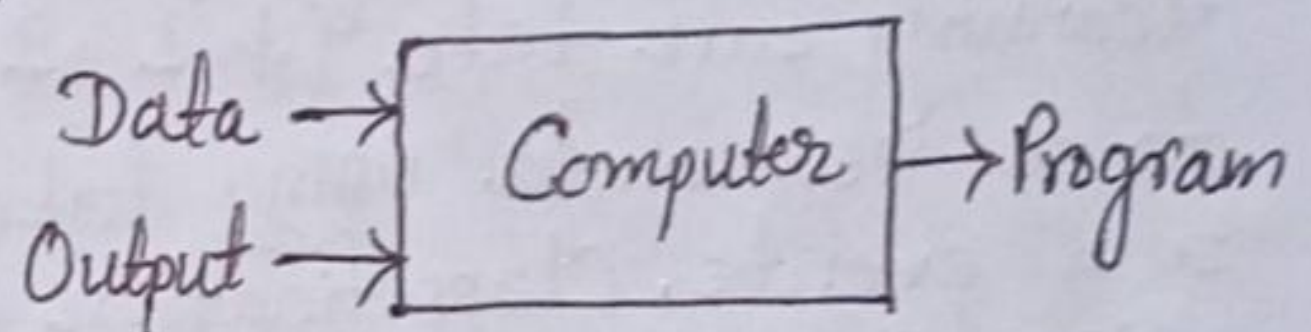
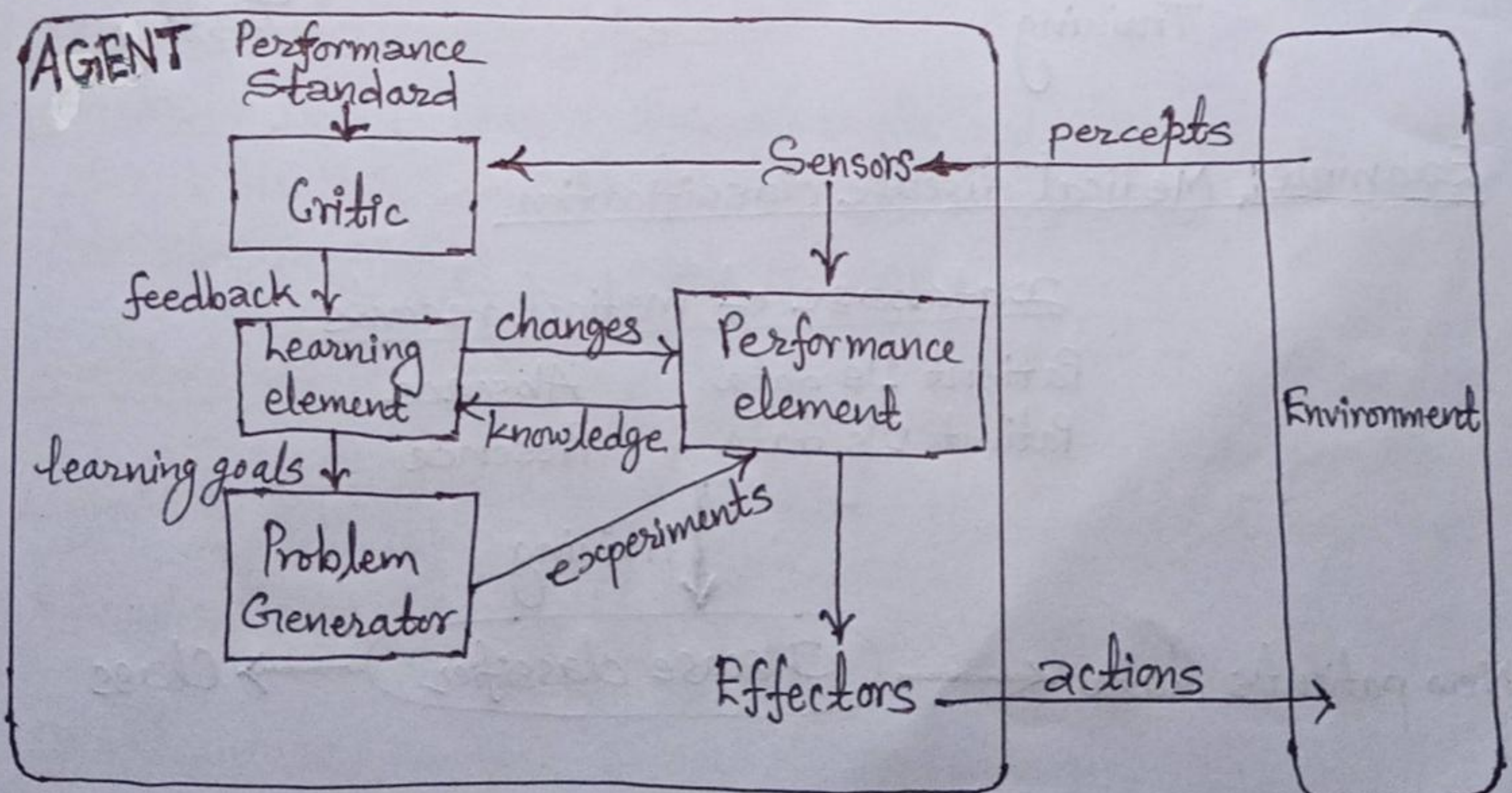


fig: Machine learning

#### ⊛. Machine learning architecture or framework (Concept of Learning):



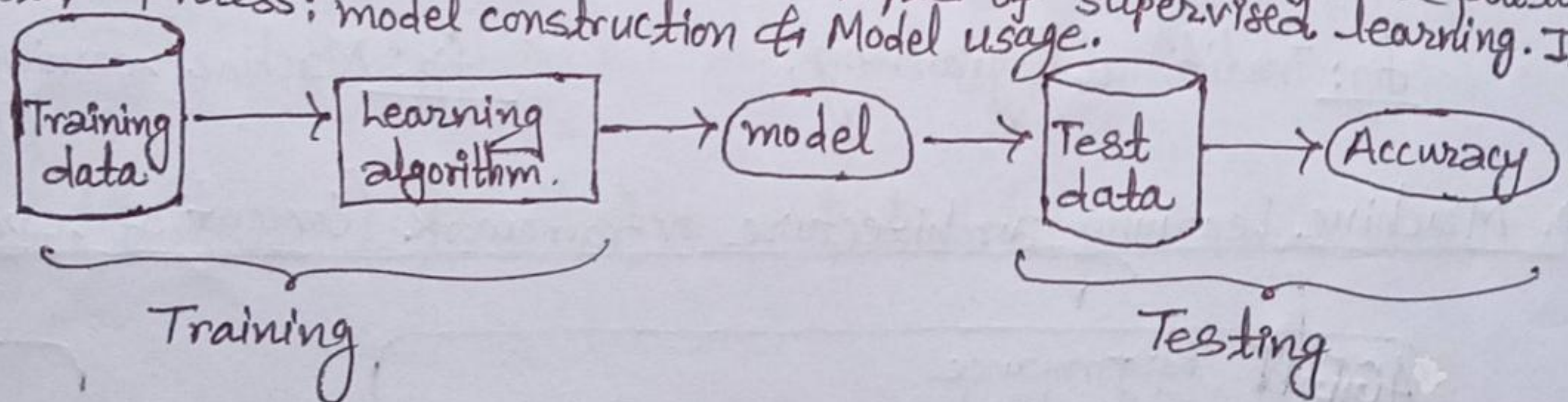
Learning agent consists of following components:



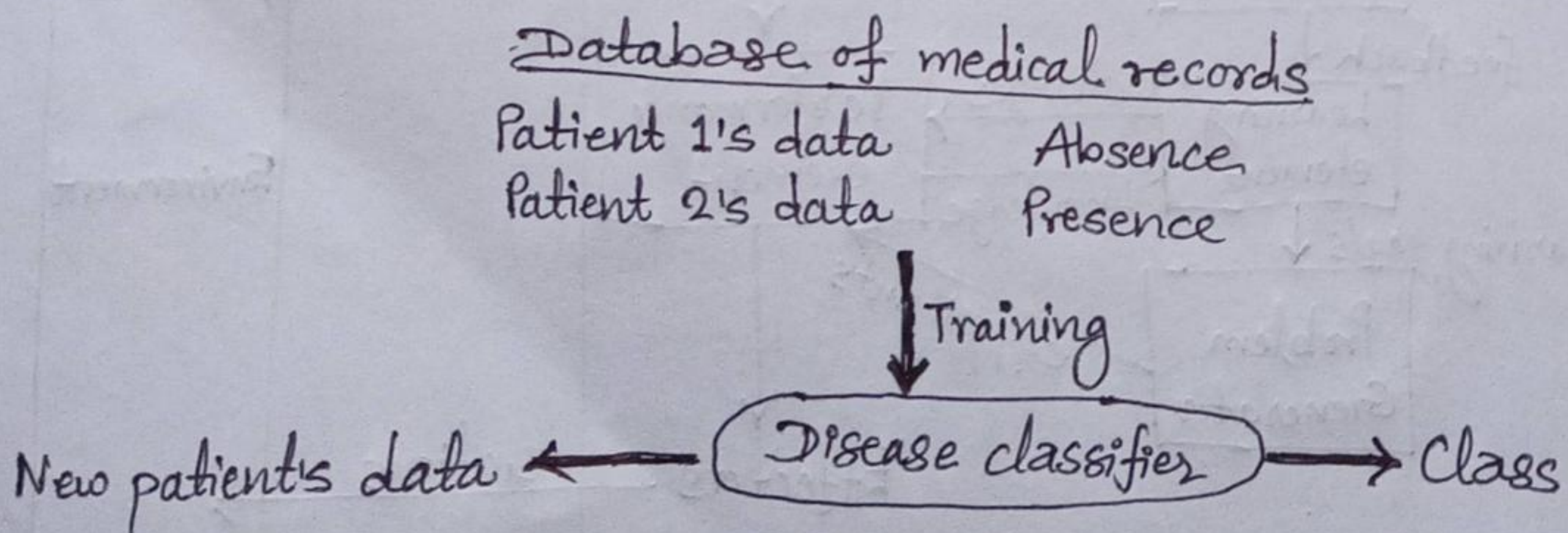
- i) Learning element → It receives and processes the input obtained from environment.
- ii) Knowledge base → It is a collection of knowledge. It updates knowledge periodically if agent is able to acquire new knowledge.
- iii) Performance element → It uses the updated knowledge base to perform some tasks or solves some problems and produces the corresponding output.
- iv) Feedback element → It receives two inputs, one from learning element and one from standard system. It is used to determine what should be done to produce the correct output.
- v) Standard system → It is a well programmed system that is able to produce the correct output.

### ⊗. Types of Machine Learning:

- i) Supervised learning → The system is supplied with a set of training examples consisting of inputs and corresponding outputs. Supervised learning can take what it has learnt in the past & apply that to a new data using labelled example to predict future patterns and events. Classification is an example of supervised learning. It is two-step process: model construction & Model usage.



### Example: Medical disease classification.





i) Unsupervised learning → It is also called predictive learning. In this learning information is used to train is neither classified nor labelled. It studies how system can infer a function to describe a hidden structure unlabelled data. This is similar to how babies learn easily in their life.

Examples: Clustering of data into similar group, Predict new target market etc.

ii) Semi-supervised learning → It falls somewhere in between supervised and unsupervised learning, since they use both labelled and unlabelled data for training. Typically a small amount of labelled data and a large amount of unlabelled data. This method are able to improve learning accuracy.

Example:- Classification of target market after searching information on GOOGLE.

iii) Reinforcement Learning → It is a type of dynamic learning that trains agent using reward & punishment. In this learning, agent learns by interacting with environment & consists of three components: Agent (learner), Environment (Agent interacts with) and Action (what agent can do).

Example: FB Newsfeed, Youtube recommendations etc.

Types of Reinforcement: (less imp)

i) Positive → It increases the strength and the frequency of the behaviour. It has a positive effect on the behaviour.

Advantages: → Maximizes performance  
→ Sustain change for long period of time.

Disadvantage: Overhead of states can diminish the results.

ii) Negative → Strengthening of a behaviour because a negative condition is stopped or avoided.

Advantage: → Increases behaviour

Disadvantage: Provides enough to meet up the minimum behaviour.



## \*. Statistical-based Learning: Naive Bayes Model:-

Basics:

for understanding  
and solving problems  
in easier way

→ Probability of event X i.e,  $P(X) = \frac{\text{No. of times X occurs}}{\text{Total no. of events in sample space}}$

i) Joint Probability: Consider two events A & B that occurs:  
 $P(A \cap B) = P(A) * P(B)$  if the events are independent.  
 $P(A \cap B) = P(A/B) * P(B)$

ii) Bayes Theorem:  $P(A/B) = \frac{P(B/A) (P(A))}{P(B)}$

write from here  
if asked in exam

### ✓ Naive Bayes Learning/Classifier:

In this model class variable C (which is to be predicted) is the root & the attribute variable  $x_i$  are the leaves. The model is called naive because that attributes are conditionally independent of each other in the given class. To calculate Naive Bayes Learning we use Bayes's theorem, which helps to calculate conditional dependent probability.

$$P(C/x_1 \dots x_n) = \alpha P(x) \prod_{i=1}^n P(x_i/C)$$

This model is used to calculate the mostly likelihood parameters or attributes of a given class.

### Applications of Naive Bayes Model:

- Real time prediction.
- Text classification.
- Recommendation system.

Example:- From the given data set decide whether to play golf or not in today's conditions:

today = (Sunny, Hot, Normal, False).



| S.N. | Outlook  | Temperature | Humidity | Wind  | Play Golf |
|------|----------|-------------|----------|-------|-----------|
| 1.   | Rainy    | Hot         | High     | False | No        |
| 2.   | Rainy    | Hot         | High     | True  | No        |
| 3.   | Overcast | Hot         | High     | False | Yes       |
| 4.   | Sunny    | Mild        | High     | False | Yes       |
| 5.   | Sunny    | Cool        | Normal   | False | Yes       |
| 6.   | Sunny    | Cool        | Normal   | True  | No        |
| 7.   | Overcast | Cool        | Normal   | True  | Yes       |
| 8.   | Rainy    | Mild        | High     | False | No        |
| 9.   | Rainy    | Cool        | Normal   | False | Yes       |
| 10.  | Sunny    | Mild        | Normal   | False | Yes       |
| 11.  | Rainy    | Mild        | Normal   | True  | Yes       |
| 12.  | Overcast | Mild        | High     | True  | Yes       |
| 13.  | Overcast | Hot         | Normal   | False | Yes       |
| 14.  | Sunny    | Mild        | High     | False | No        |

Solution:

For Outlook:

|          | Yes | No | P(Yes)        | P(No)         |
|----------|-----|----|---------------|---------------|
| Sunny    | 3   | 2  | $\frac{3}{9}$ | $\frac{2}{5}$ |
| Overcast | 4   | 0  | $\frac{4}{9}$ | 0             |
| Rainy    | 2   | 3  | $\frac{2}{9}$ | $\frac{3}{5}$ |
| Total    | 9   | 5  |               |               |

For Temperature:

|       | Yes | No | P(Yes)        | P(No)         |
|-------|-----|----|---------------|---------------|
| Hot   | 2   | 2  | $\frac{2}{9}$ | $\frac{2}{5}$ |
| Cool  | 3   | 1  | $\frac{3}{9}$ | $\frac{1}{5}$ |
| Mild  | 4   | 2  | $\frac{4}{9}$ | $\frac{2}{5}$ |
| Total | 9   | 5  |               |               |



For Humidity:

|        | Yes | No | P(Yes)        | P(No)         |
|--------|-----|----|---------------|---------------|
| High   | 3   | 4  | $\frac{3}{9}$ | $\frac{4}{5}$ |
| Normal | 6   | 1  | $\frac{6}{9}$ | $\frac{1}{5}$ |
| Total  | 9   | 5  |               |               |

For Wind:

|       | Yes | No | P(Yes)        | P(No)         |
|-------|-----|----|---------------|---------------|
| True  | 3   | 2  | $\frac{3}{9}$ | $\frac{2}{5}$ |
| False | 6   | 3  | $\frac{6}{9}$ | $\frac{3}{5}$ |
| Total | 9   | 5  |               |               |

Since today's conditions are Sunny, Hot, Normal, False only according to the question

Now,

$$\begin{aligned}
 P(\text{Yes/Today}) &= P(\text{Sunny/Yes}) \times P(\text{Hot/Yes}) \times P(\text{Normal/Yes}) \times P(\text{False/Yes}) \\
 &\quad \times P(\text{Yes}) \\
 &= \frac{3}{9} \times \frac{2}{5} \times \frac{6}{9} \times \frac{6}{9} \times \frac{9}{14} \quad \leftarrow \frac{\text{Total no. of Yes}}{\text{Total Yes} + \text{Total No}} \\
 &= 0.0212
 \end{aligned}$$

$$\begin{aligned}
 P(\text{No/Today}) &= P(\text{Sunny/No}) \times P(\text{Hot/No}) \times P(\text{Normal/No}) \times P(\text{False/No}) \\
 &\quad \times P(\text{No}) \\
 &= \frac{2}{5} \times \frac{2}{5} \times \frac{1}{5} \times \frac{3}{5} \times \frac{5}{14} \\
 &= 0.0069.
 \end{aligned}$$

Since, sum of probabilities must be equal to 1. So, we have to normalize.

$$\begin{aligned}
 \text{i.e., } P(\text{Yes/Today}) &= \frac{P(\text{Yes/Today})}{P(\text{Yes/Today}) + P(\text{No/Today})} \\
 &= \frac{0.0212}{0.0212 + 0.0069} \\
 &= 0.7544
 \end{aligned}$$

$$\begin{aligned}
 \& P(\text{No/Today}) &= \frac{P(\text{No/Today})}{P(\text{No/Today}) + P(\text{Yes/Today})} = \frac{0.0069}{0.0069 + 0.0212} = 0.2456
 \end{aligned}$$

$\therefore P(\text{Yes/Today}) > P(\text{No/Today})$ . So, they can play golf in today's conditions.



## ⊗ Learning by Genetic Algorithm (GA):

Genetic Algorithms (GAs) are adaptive heuristic search algorithm that belongs to the larger part of evolutionary algorithms. It is based on the idea of natural selection & genetics. The historical data are provided to find out better solution or simply GA's are used to generate high quality solutions for optimization and search problem.

Genetic Algorithms (GA's) simulate the process of natural selection which means those species who can adapt to change their environment are able to survive & reproduce next generation (fittest survival). Each generation consists of a population of individuals and each individual represents a point in search space & possible solutions which are represented as string of characters/integers/floats/bits.

### Operations in Genetic Algorithm:

- i) Reproduction/Selection → This operator provides the performance to the individuals with good fitness score and allow them to pass their genes to the successive generation. To select the parents the techniques are round wheel, tournament selection, rank selection etc.
- ii) Crossover → This represents mating between two individuals. Two individual are selected using selection operators and crossover sites are chosen randomly then genes at the crossover sites are exchanged thus, it creates completely new offsprings.

Example:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 5 | 8 | 9 | 4 | 2 | 3 | 5 | 7 | 5 | 8 |
|---|---|---|---|---|---|---|---|---|---|

→ 

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 7 | 5 | 8 |
|---|---|---|---|---|---|---|---|---|

Child after crossover.

The popular cross overs are one-point crossover, multipoint crossover & uniform crossover.



iii) Mutation → The key idea is to insert random genes in offspring to maintain the diversity in population to avoid the premature convergence.  
Example:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 1 |
|---|---|---|---|---|---|

 → 

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 1 |
|---|---|---|---|---|---|

  
Before mutation                      After mutation.

iv) Fitness function → It is the function which takes a candidate solution to the problem as input & produce output, determines how good or fit candidate is.

### Genetic Algorithm:

- Randomly initialize population.
- Determine fitness of population.
- Until convergence repeat:
  - Select parents from population.
  - Crossover and generate new population.
  - Perform mutation on new population.
  - Calculate fitness for new population.

### Applications of GA's:

- Recurrence Neural Networks.
- Mutation Testing
- Learning Fuzzy rule.



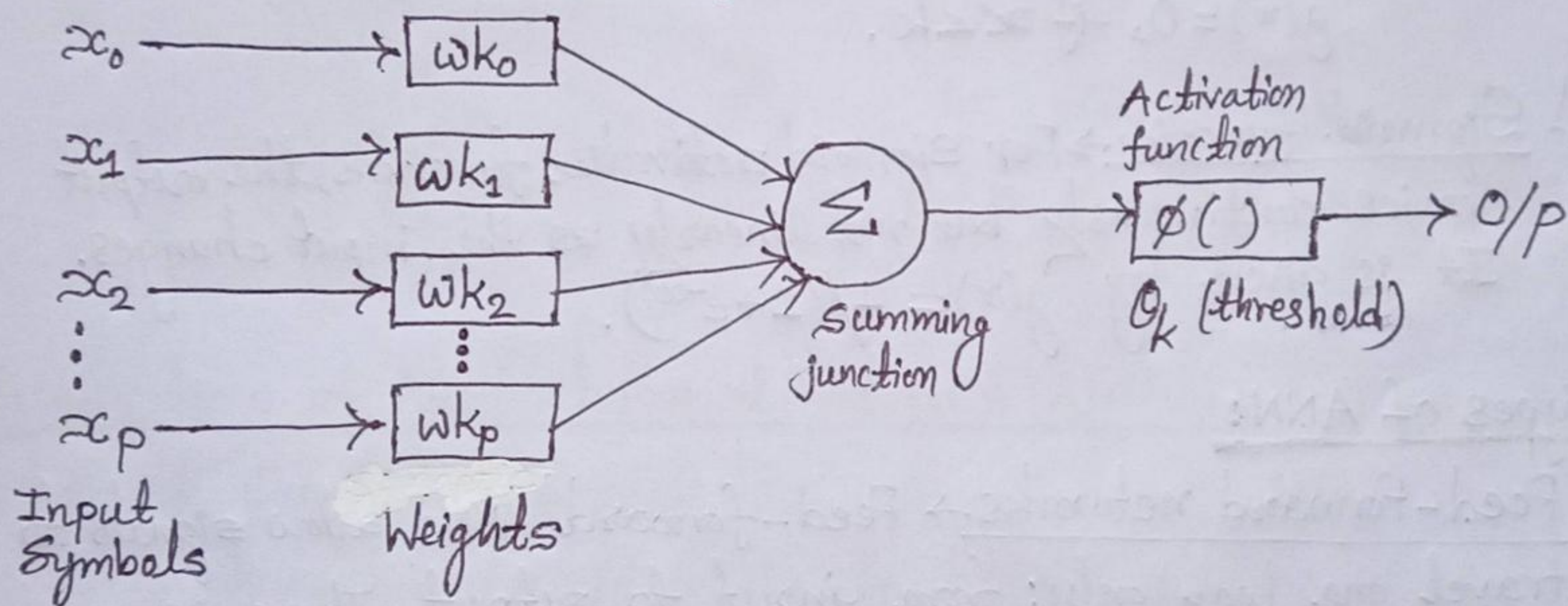
## \* Learning with Neural Networks:

### Artificial Neural Network (ANN):-

ANN is the non-linear parallelly distributed & highly connected network having capacity of adaptivity, self-organization, fault tolerance & very large scale integration (VLSI) implementation of neurons. Neural networks are composed of simple elements operating in parallel. These elements are inspired by biological normal system. The network function is determined by connecting neurons in different levels & we can train them to perform some task by assigning some values to the connection called weight.

The goal of ANN is to solve problems in the same way that human mind does. ANN are widely used in computer vision, speech recognition, medical diagnosis, face recognition, signature verification etc.

### \* Mathematical Model of ANN:



Inputs: Numerical values  $x_1, x_2, x_3 \dots x_p$  are the input in neurons.

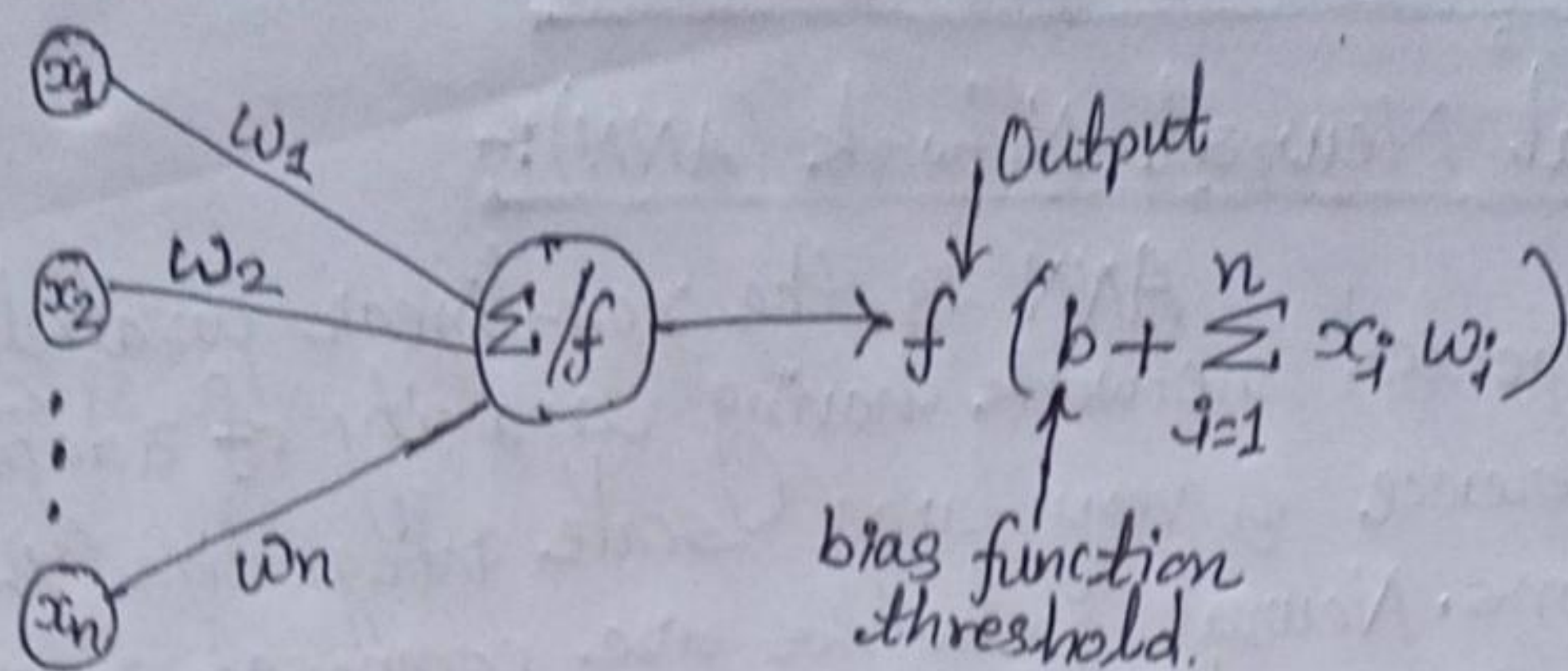
Bias weight: Weight of all neurons determined by threshold (i.e, predefined value to make thing correct).

Activation function: The process of deciding output after aggregating input is called aggregation function. Each unit first computes a weighted sum of its inputs:  

$$in_i = \sum_{k=0}^n w_{k,i} \cdot x_k$$
 , then it applies activation function  $g$  to this sum to derive the output:  $O_i = g(in_i) = g(\sum_{k=0}^n w_{k,i} \cdot x_k)$ .



where,  $g$  is activation function.



### \* Types of Activation Functions:

i) Linear function → For linear activation functions, the output activity is proportional to the total weighted output. This function is given by  $g(x) = kx + c$ , where  $k$  &  $c$  are constant.

ii) Threshold function → For threshold functions, the output are set at one of the two levels, depending on whether the total input is greater than or less than some threshold value.

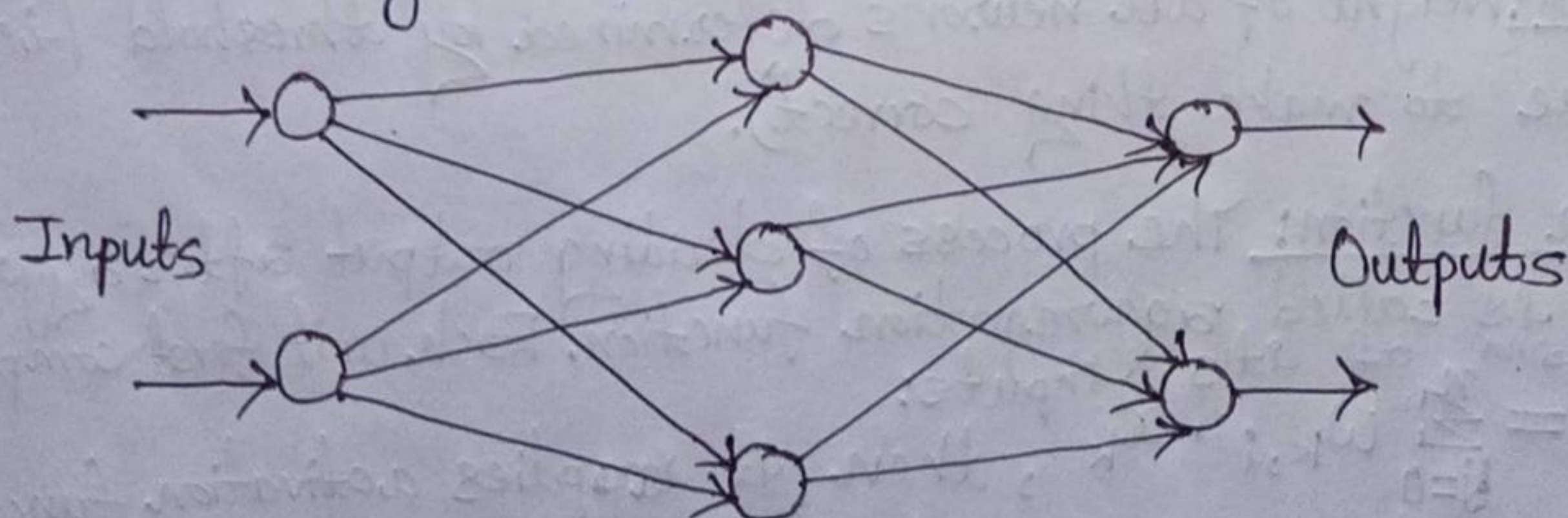
$$g(x) = 1, \text{ if } x \geq k$$

$$g(x) = 0, \text{ if } x < k.$$

iii) Sigmoid function → For Sigmoid activation functions, the output varies continuously but not linearly as the input changes. It is given by  $g(x) = 1/(1 + e^{-x})$ .

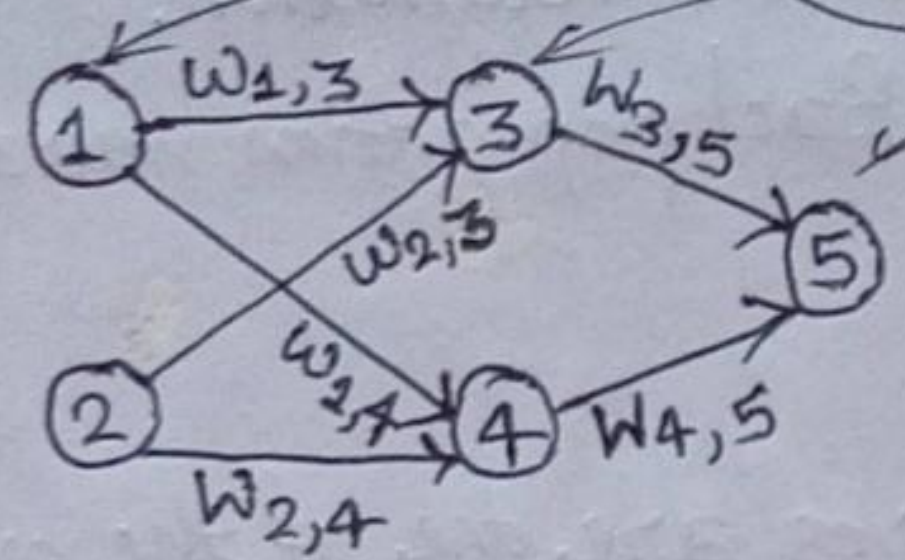
### \* Types of ANN:

i) Feed-forward networks → Feed-forward ANNs allow signals to travel one way only: from input to output. There is no feedback (loops). i.e., the output of any layer does not affect the same layer. This types of ANNs are extensively used in pattern recognition.





Example:



first layer  $a_1$   
second layer  $a_2$   
third layer  $a_3$

output  $z_1$   $z_2$   $z_3$

$$g_3 = g(w_{1,3}a_1 + w_{2,3}a_2)$$

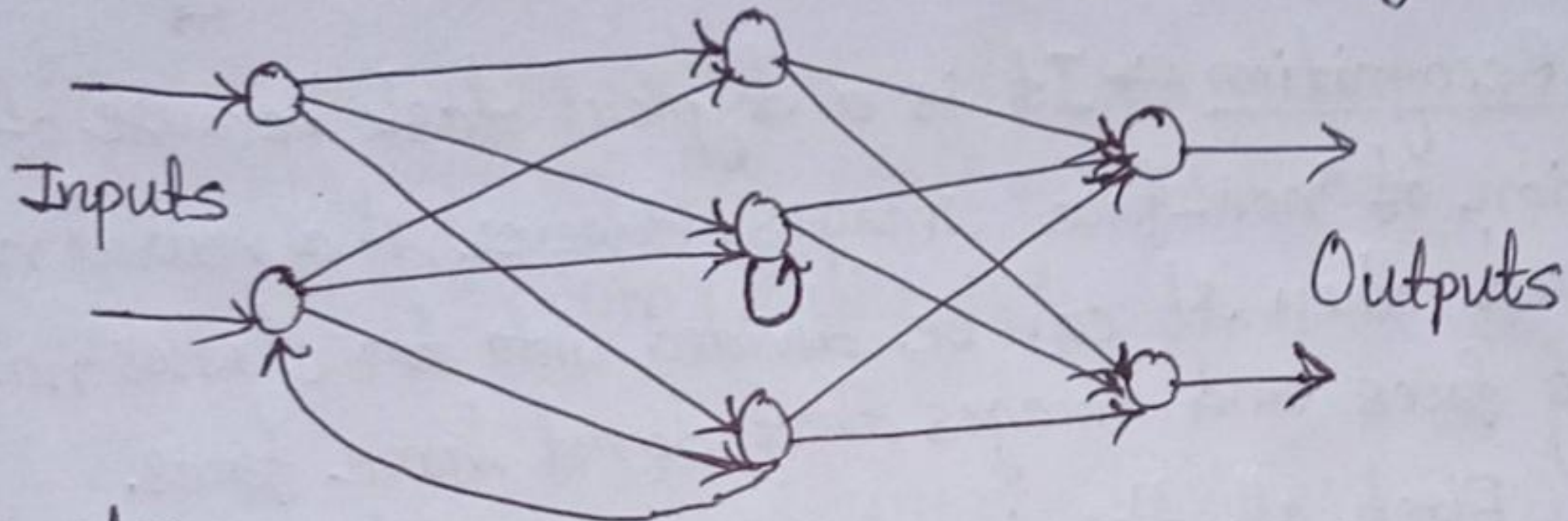
$$g_4 = g(w_{1,4}a_1 + w_{2,4}a_2)$$

$$g_5 = g(w_{3,5}a_3 + w_{4,5}a_4)$$

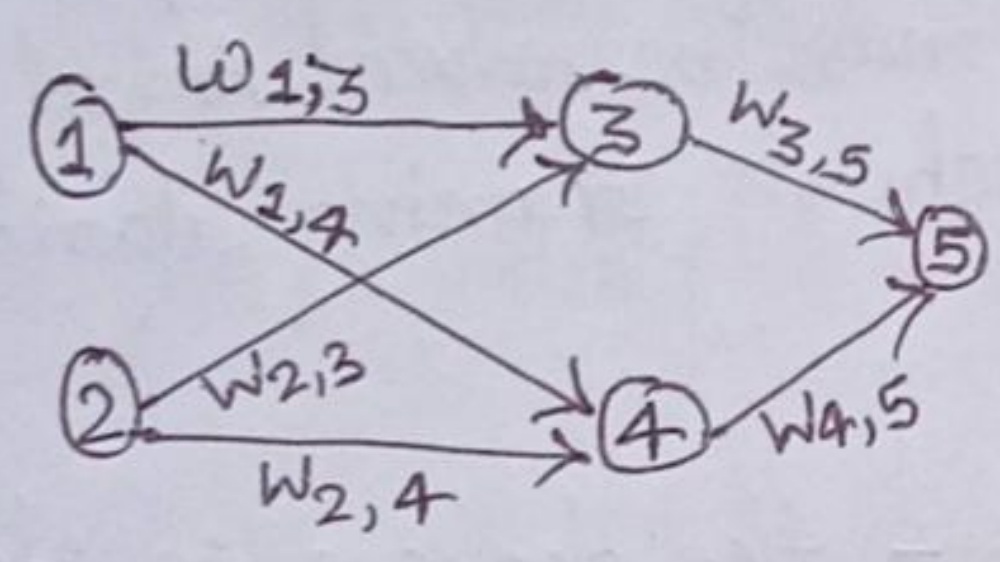
Note: If fig. contains 2 layer i.e.  $a_1, a_2$  then it is single layer. If contains more than two it is multilayer.

Examples described here are multilayer since contains more than 2 i.e. 3 layers

Feedback networks (Recurrent networks) → Feedback networks can have signals travelling in both directions by introducing loops in the network.



Example:



Here,

$$g_5 = g(w_{3,5}a_3 + w_{4,5}a_4)$$

$$= g(w_{3,5}g(w_{1,3}a_1 + w_{2,3}a_2) + g(w_{1,4}a_1 + w_{2,4}a_2))$$

## Applications of Artificial Neural Networks (ANNs):

Speech Recognition → Speech occupies a prominent role in human-human interaction. Therefore, it is natural for people to expect speech interfaces with computers. ANN has made great progress in this field. Following ANNs have been used for speech recognition:-

- Multilayer networks
- Multilayer networks with recurrent connections.
- Kohonen self-organizing feature map.

Character Recognition → It is an interesting problem which falls under the general area of Pattern Recognition. Many neural networks have been developed for automatic recognition of handwritten characters, either letters or digits. Following are the



Some ANNs which have been used for character recognition:

→ Multilayer neural networks such as Backpropagation neural networks.

→ Neocognitron.

iii) Signature Verification Application → Signatures are one of the most useful ways to authorize and authenticate a person in legal transactions. Signature verification technique is a non-vision based technique. The trained neural network will classify the signature as being genuine or forged under the verification stage.

iv) Human Face Recognition → It is a typical task because of the characterization of "non-face" images. However, if a neural network is well trained, then it can be divided into two classes namely images having faces and images that do not have faces.

First, all the input images must be processed. Then, the dimensionality of that image must be reduced. And at last it must be classified using neural network training algorithm.

### ⊗ Learning by Training ANN:

Learning means to adopt the changes in itself when there is change in environment. In complex system of ANN, after learning it changes its internal structure based on the information passing through it.

Learning is important in ANN because of some changes occurring in environment then ANN must change its inputs, outputs, activation function & Weight. To change the internal structure of ANN there are various methods (i.e, learning rules) which are as follows;

1) Hebbian Learning rule: Hebbian learning rule is introduced by Donald Hebbian in 1949. It is also called unsupervised feed forward learning. This rule basically states that, if neuron  $i$  is near enough to excite neuron  $j$  and repeatedly participates in its activation, the synaptic connection between these two neurons is strengthened and neuron  $j$  becomes more sensitive to

i.e, made stronger



from neuron  $i$ .

Hebb's law can be represented in the form of two rules:

- i) If two neurons on either side of a connection are activated synchronously, then the weight of that connection is increased.
- ii) If two neurons on either side of a connection are activated asynchronously, then the weight of that connection is decreased.

### Hebb's Algorithm:

Step 1: Initialize all weights and bias to 0.

Step 2: Given a training input  $s$ , with its target output  $t$ , set the activations of the input units:  $x_i = s_i$

Step 3: Set the activation of the output unit to the target value:  $y = t$

Step 4: Adjust the weights:  $w_i(\text{new}) = w_i(\text{old}) + x_i y$

Step 5: Adjust the bias (just like the weights):  $b(\text{new}) = b(\text{old}) + y$

Problem: Construct a Hebb Net which performs like an AND function.

Solution:

Let us construct truth table of AND gate as follows:-

| INPUT |       |       |     | TARGET |     |
|-------|-------|-------|-----|--------|-----|
|       | $x_1$ | $x_2$ | $b$ |        | $y$ |
| $X_1$ | -1    | -1    | 1   | $Y_1$  | -1  |
| $X_2$ | -1    | 1     | 1   | $Y_2$  | -1  |
| $X_3$ | 1     | -1    | 1   | $Y_3$  | -1  |
| $X_4$ | 1     | 1     | 1   | $Y_4$  | 1   |

There are 4 training samples, so there will be 4 iterations.

Step 1: Set weights and bias to zero,  $w = [0 \ 0 \ 0]$  and  $b = 0$ .

Step 2: Set the activations of input units  $x_i = s_i$  for  $i = 1$  to 4.

$$X_1 = [-1 \ -1 \ 1]$$

$$X_2 = [-1 \ 1 \ 1]$$

$$X_3 = [1 \ -1 \ 1]$$

$$X_4 = [1 \ 1 \ 1]$$

i.e.  $X_1 = [x_1 \ x_2 \ b]$

b always 1  
if  $x_1, x_2$  are 1  
AND  
gate with same bias  
to represent OR gate



Step 3: Output value is set to  $y=t$ .

Step 4: Adjust (or Modify) weights using Hebbian Rule:

1<sup>st</sup> iteration:  $w(\text{new}) = w(\text{old}) + x_1 y_1$

$$= [0 \ 0 \ 0] + [-1 \ -1 \ 1] \cdot [-1]$$

$$= [1 \ 1 \ -1]$$

2<sup>nd</sup> iteration:  $w(\text{new}) = w(\text{old}) + x_2 y_2$

for 2<sup>nd</sup> iteration,  
final wts of 1<sup>st</sup> iteration  
will be used & so on...

$$= [1 \ 1 \ -1] + [-1 \ 1 \ 1] \cdot [-1]$$

$$= [2 \ 0 \ -2]$$

3<sup>rd</sup> iteration:  $w(\text{new}) = w(\text{old}) + x_3 y_3$

$$= [2 \ 0 \ -2] + [1 \ -1 \ 1] \cdot [-1]$$

$$= [1 \ 1 \ -3]$$

4<sup>th</sup> iteration:  $w(\text{new}) = w(\text{old}) + x_4 y_4$

$$= [1 \ 1 \ -3] + [1 \ 1 \ 1] \cdot [1]$$

$$= [2 \ 2 \ -2]$$

So, the final weight matrix is  $[2 \ 2 \ -2]$ .

$\{x_1, x_2, b\}$  form

Testing the Network:

Step 5

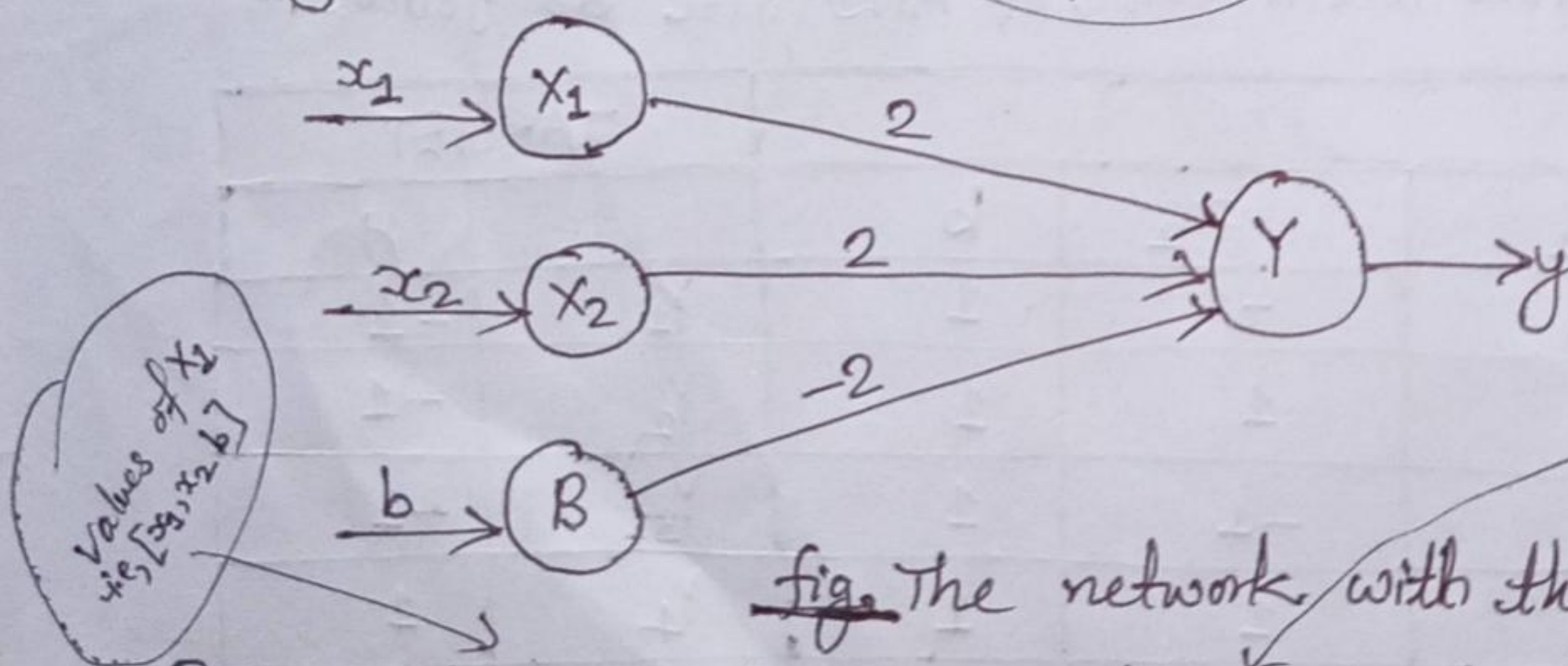


fig. The network with the final weights.

$$\text{For } x_1 = -1, x_2 = -1, b = 1, Y = (-1)(2) + (-1)(2) + (1)(2) = -6$$

$$\text{For } x_1 = -1, x_2 = 1, b = 1, Y = (-1)(2) + (1)(2) + (1)(2) = -2$$

$$\text{For } x_1 = 1, x_2 = -1, b = 1, Y = (1)(2) + (-1)(2) + (1)(2) = -2$$

$$\text{For } x_1 = 1, x_2 = 1, b = 1, Y = (1)(2) + (1)(2) + (1)(2) = 2$$

i.e., first 3  
values negative  
and final only  
positive

The results are all compatible with the original table.

Decision Boundary:

$$\text{We have } 2x_1 + 2x_2 - 2b = y$$

$$\text{Now replacing } y \text{ with } 0, \text{ we get } 2x_1 + 2x_2 - 2b = 0$$

$$\text{Since, bias, } b = 1, \text{ so, } 2x_1 + 2x_2 - 2(1) = 0 \quad \text{i.e., } 2(x_1 + x_2) = 2$$



From  $2(x_1 + x_2) = 2$ , we get final equation,  $x_2 = -x_1 + 1$ .

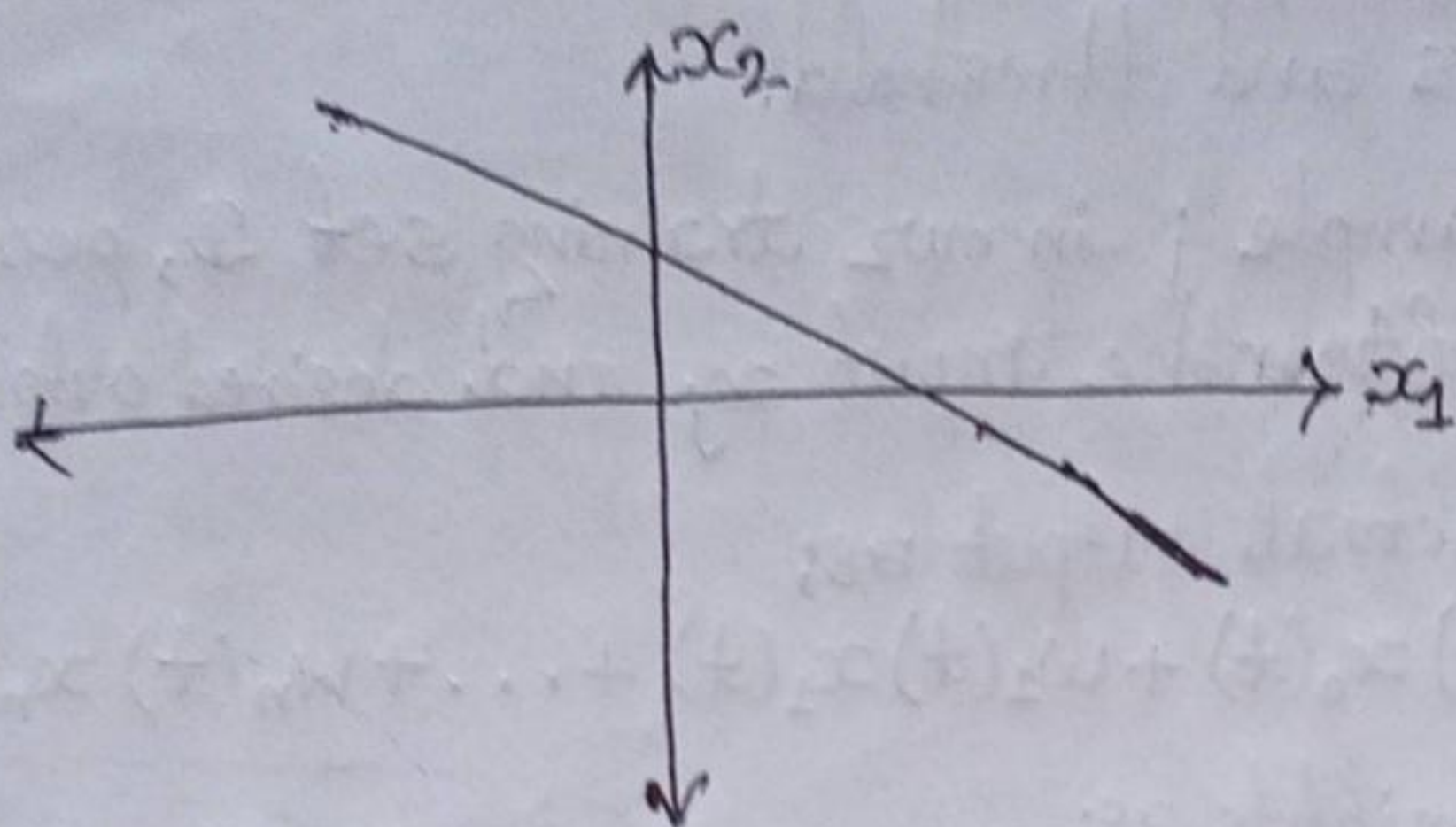


fig. Decision Boundary of AND Function.

less imp but once have a look similar to algorithm only use of  $i$  to  $n$  braces

### Pseudocode for Hebb's Net (Supervised)

```

Initialization; for  $i=1$  to  $n$ :  $\{b=0, w_i=0\}$ .
For each of the training samples  $s$ : to do
{
  /*  $s$  is the input pattern,  $t$  the target output of the sample */
  for  $i=1$  to  $n$   $\{x_i = s_i\}$  /* set to input units */
   $y = t$  /* set  $y$  to the target */
  for  $i=1$  to  $n$  {
     $w_i = w_i + x_i * y$ 
  } /* update weight */
   $b = b + x_i * y$  /* update bias */
}

```

### 2) Perceptron Learning:

Perceptron learning is used in supervised learning rule to classify the data in two classes. It consist of a single neuron with arbitrary value 1 or 0 depending on threshold & also consist of bias weight. Training patterns are presented to the network's inputs; the output is computed.

#### Variables used:

\*  $y = f(x)$  denotes the output for an input vector  $x$ .

\*  $D = (x_1, d_1), (x_2, d_2), \dots, (x_n, d_n)$  is the training set of  $n$  samples. where,  $x_j \rightarrow$  input vector  
 $d_j \rightarrow$  desired output for  $x_j$ .



### Algorithm:

Step 1: Initialize weights and threshold.

Step 2: For each example  $j$  in our training set  $D$ , perform following steps 3 & 4 over the input  $x_j$  and desired output  $d_j$ .

Step 3: Calculate the actual output as;

$$y(t) = g[w_0(t)x_0(t) + w_1(t)x_1(t) + \dots + w_n(t)x_n(t)]$$

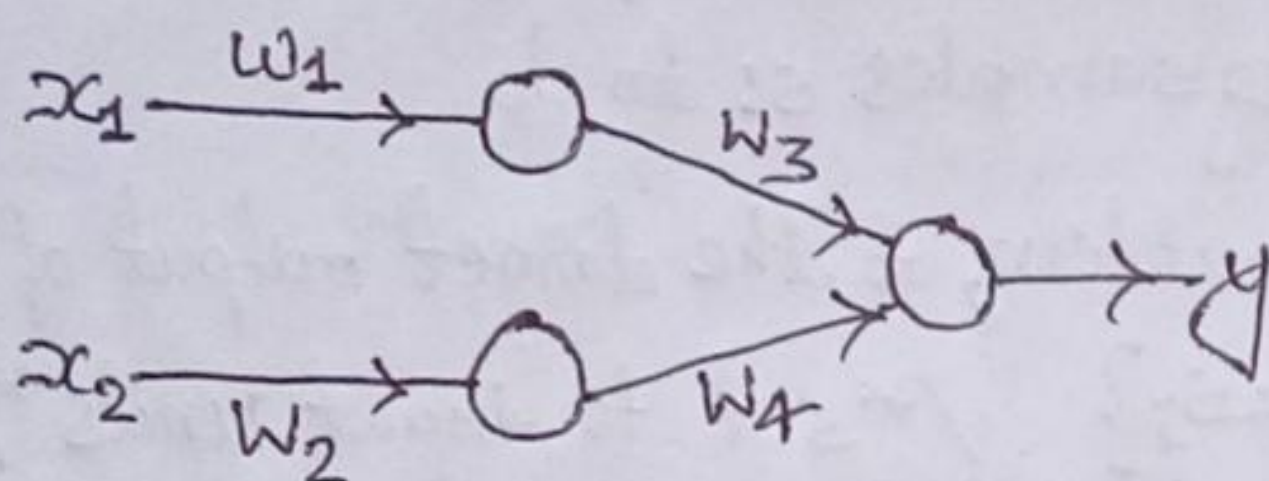
Step 4: Update the weights as;

$$w_i(t+1) = w_i(t) + \alpha [d(t) - y(t)] x_i(t)$$

where,  $0 \leq \alpha \leq 1$  (learning rate)

i.e.,  $y(t) = g[\sum w_i(t) \cdot x_i(t)]$

### Example:



Given,

desired output  $d(t)$ .

Actual output is computed as;

$$y(t) = g(y_1 w_3(t) + y_2 w_4(t))$$

$$\text{where, } y_1 = g(x_1 w_1(t))$$

$$y_2 = g(x_2 w_2(t))$$

Now update weight as;

$$w_1(t+1) = w_1(t) + \alpha [d(t) - y(t)] x_1(t)$$

$$w_2(t+1) = w_2(t) + \alpha [d(t) - y(t)] x_2(t)$$

$$w_3(t+1) = w_3(t) + \alpha [d(t) - y(t)] y_1(t)$$

$$w_4(t+1) = w_4(t) + \alpha [d(t) - y(t)] y_2(t)$$

After updating weights, we recompute  $y(t)$ . If the desired solution is obtained then stop otherwise we will iterate again.



### 3) Back-propagation learning:-

It is a supervised learning method, and is an implementation of Delta rule. It requires a teacher that can calculate the desired output for any given input. It is most useful for feed-forward networks. Back propagation requires that the activation function used by the artificial neurons (or nodes) is differentiable.

#### Algorithm:

1. Randomly choose the initial weight.
2. Compute output ( $y$ ) from neural network;

$$y = g(\sum x_i w_i) \text{ where, } g(x) = \frac{1}{1+e^{-x}} \text{ is sigmoid function.}$$

3. Compute error  $S = t - y$ ; where  $t$  is targeted output and  $y$  is actual output.

4. Propagate this  $S$  error back to layer of neural network and compute error at each layer as;

$$\delta_i = \sum w_{ij} \delta_j$$

5. Update the weights as;

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha \delta_j \frac{d(y_i)}{dy_i} x_i$$

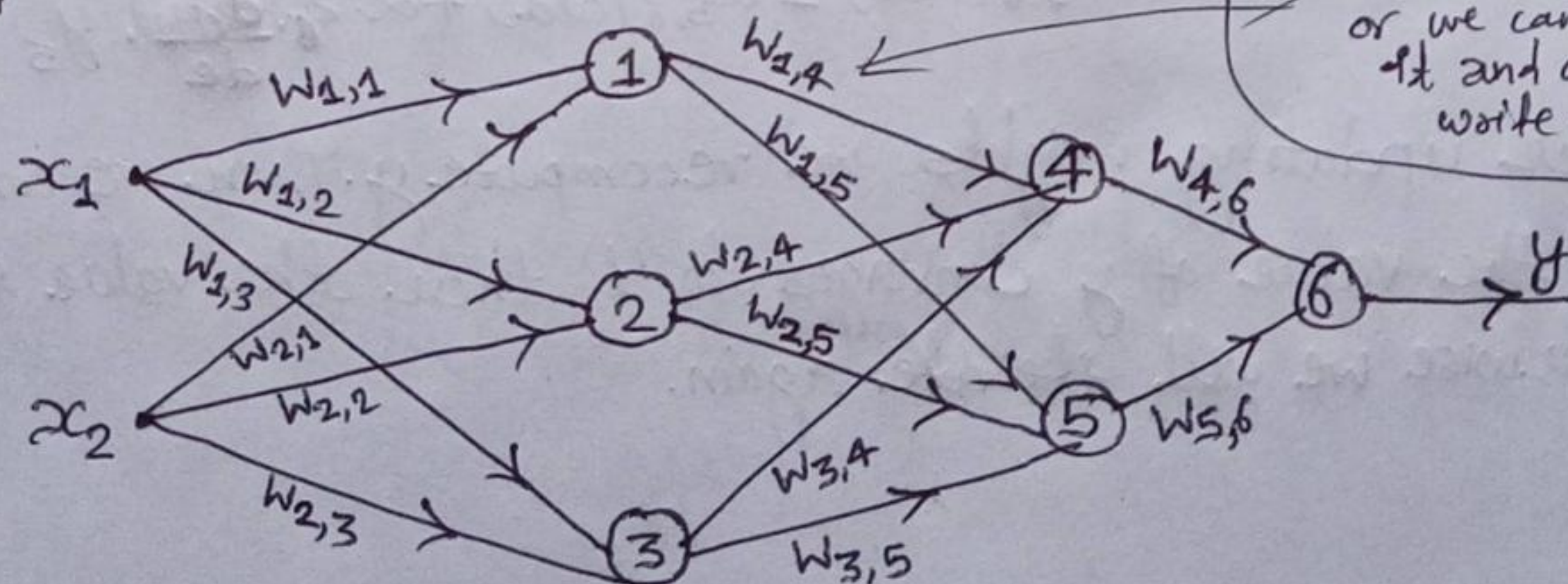
where,  $\frac{d(y_i)}{dy_i}$  is derivative of  $y_i$  w.r.t.  $e$ .

$\alpha$  is learning rate  $0 \leq \alpha \leq 1$

$x_i$  is input to node.

6. Repeat step 2 to 5 until  $y$  converges to  $t$ .

#### Example 1:



We can use comma or we can replace it and also simply write as  $w_{14}$



$$y = g(y_4 w_{4,6} + y_5 w_{5,6})$$

where,

$$y_4 = g(y_1 w_{1,4} + y_2 w_{2,4} + y_3 w_{3,4})$$

$$y_5 = g(y_1 w_{1,5} + y_2 w_{2,5} + y_3 w_{3,5})$$

where,

$$y_1 = g(x_1 w_{1,1} + x_2 w_{2,1})$$

$$y_2 = g(x_1 w_{1,2} + x_2 w_{2,2})$$

$$y_3 = g(x_1 w_{1,3} + x_2 w_{2,3})$$

Given target  $t$ , Compute  $S = t - y$ .  
Now calculate backpropagate error as;

$$\delta_4 = w_{4,6} * S$$

$$\delta_5 = w_{5,6} * S$$

$$\delta_1 = w_{1,4} \delta_4 + w_{1,5} \delta_5$$

$$\delta_2 = w_{2,4} \delta_4 + w_{2,5} \delta_5$$

$$\delta_3 = w_{3,4} \delta_4 + w_{3,5} \delta_5$$

Now update weight as;

$$w_{1,1}(\text{new}) = w_{1,1}(\text{old}) + \alpha \delta_1 \frac{d(y_1)}{de} \cdot x_1$$

$$w_{1,2}(\text{new}) = w_{1,2}(\text{old}) + \alpha \delta_2 \frac{d(y_2)}{de} \cdot x_1$$

$$w_{1,3}(\text{new}) = \dots (\text{similarly})$$

$$w_{2,1}(\text{new}) = w_{2,1}(\text{old}) + \alpha \delta_1 \frac{d(y_1)}{de} \cdot x_2$$

$\vdots$

$$w_{2,5}(\text{new}) = w_{2,5}(\text{old}) + \alpha \delta_5 \frac{d(y_5)}{de} \cdot y_2$$

$$w_{3,4}(\text{new}) = w_{3,4}(\text{old}) + \alpha \delta_4 \frac{d(y_4)}{de} \cdot y_3$$

$\vdots$

$$w_{4,6}(\text{new}) = w_{4,6}(\text{old}) + \alpha S \frac{dy}{de} \cdot y_4$$

$$w_{5,6}(\text{new}) = w_{5,6}(\text{old}) + \alpha S \frac{dy}{de} \cdot y_5$$

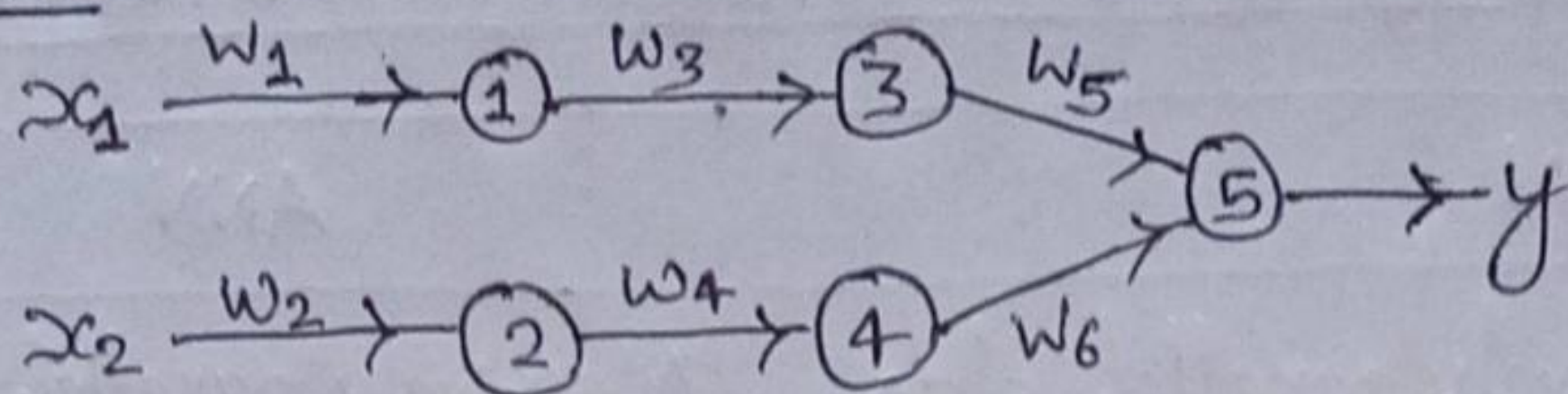
if confusion once  
look and compare  
with algorithm.

$w_{1,3}$   $\frac{dy}{de}$   $\delta_1$   $x_1$   
As  $w_{1,1}$ ,  
 $w_{1,2}$  similarly we  
can do  $w_{1,3}$  easily  
do yourself

After updating weights, we recompute  $y$ . Then we compute  $S = t - y$ .  
If the value of  $y$  converges to  $t$  then the value is fixed up  
otherwise we will iterate again.



Example 2:



$$y = g(y_3 w_5 + y_4 w_6)$$

where,  $y_3 = g(y_1 w_3)$

$$y_4 = g(y_2 w_4)$$

where,

$$y_1 = g(x_1 w_1)$$

$$y_2 = g(x_2 w_2)$$

given target  $t$ , compute  $S = t - y$ .

Now compute backpropagate error as;

$$\delta_3 = w_5 \cdot S$$

$$\delta_4 = w_6 \cdot S$$

$$\delta_1 = w_3 \cdot \delta_3$$

$$\delta_2 = w_4 \cdot \delta_4$$

Now update weight as;

$$w_5(\text{new}) = w_5(\text{old}) + \alpha \cdot S \cdot \frac{dy}{de} \cdot y_3$$

$$w_6(\text{new}) = w_6(\text{old}) + \alpha \cdot S \cdot \frac{dy}{de} \cdot y_4$$

$$w_3(\text{new}) = w_3(\text{old}) + \alpha \cdot \delta_3 \cdot \frac{d(y_3)}{de} \cdot y_1$$

$$w_4(\text{new}) = w_4(\text{old}) + \alpha \cdot \delta_4 \cdot \frac{d(y_4)}{de} \cdot y_2$$

$$w_1(\text{new}) = w_1(\text{old}) + \alpha \cdot \delta_1 \cdot \frac{d(y_1)}{de} \cdot x_1$$

$$w_2(\text{new}) = w_2(\text{old}) + \alpha \cdot \delta_2 \cdot \frac{d(y_2)}{de} \cdot x_2$$

After updating weights, we recompute  $y$ . Then we compute  $S = t - y$ . If the value of  $y$  converges to  $t$  then the value is fixed up otherwise we will iterate again.



## ⊗ Biological Neural Networks (BNN) vs. Artificial Neural Networks (ANN):

| Criteria            | BNN                                              | ANN                                                                                  |
|---------------------|--------------------------------------------------|--------------------------------------------------------------------------------------|
| i) Processing       | Massively parallel, slow but superior than ANN   | Massively parallel, fast but inferior than BNN                                       |
| ii) Size            | $10^{11}$ neurons and $10^{15}$ interconnections | $10^2$ to $10^4$ nodes (mainly depends on type of application and network designer). |
| iii) Learning       | They can tolerate ambiguity.                     | Very precise, structured and formatted data is required to tolerate ambiguity        |
| iv) Fault tolerance | Performance degrades with even partial damage.   | It is capable of robust performance, hence has the potential to be fault tolerant.   |
| v) Storage Capacity | Stores the information in the synapse.           | Stores the information in continuous memory locations.                               |

## ⊗ Supervised Learning vs. Unsupervised Learning:

| Supervised Learning                                                                     | Unsupervised Learning                                                                            |
|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| i) Supervised learning algorithms are <del>used</del> trained using labeled data.       | i) Unsupervised learning algorithms are trained using unlabeled data.                            |
| ii) Supervised learning model predicts the output.                                      | ii) Unsupervised learning model finds the hidden patterns in data.                               |
| iii) In supervised learning, input data is provided to the model along with the output. | iii) In <del>unsupervised</del> learning, only input data is provided to the model.              |
| iv) Supervised learning needs supervision to train the model.                           | iv) Unsupervised learning does not need any supervision to train the model.                      |
| v) Supervised learning model produces an accurate result.                               | v) Unsupervised learning model may give less accurate result as compared to supervised learning. |